Plagiarism Detection System – Project Report

1. Overview of the Current Project

This project focuses on building an intelligent plagiarism detection system using machine learning and NLP techniques. The system is designed to detect various levels of plagiarism across documents by leveraging the PAN Plagiarism Corpus dataset.

Current Dataset

- PAN Plagiarism Corpus 2009
- External Analysis Corpus
- Intrinsic Analysis Corpus

What the Current Version Does

- Uses PAN dataset for training/testing
- Basic preprocessing of text
- Likely supports similarity-based detection (if implemented)

---

2. Problem Statement

Traditional plagiarism detection systems rely heavily on exact matching or shallow similarity measures. These fail in cases of: - Paraphrasing - Structural rewriting - Synonym replacement - Cross-format plagiarism (PDF, DOCX, TXT)

Goal

To build a robust AI-powered system that: - Accepts multiple file formats (PDF, DOCX, TXT) - Detects plagiarism levels (low, medium, high) - Generates structured reports

---

3. System Architecture (High-Level)

```
User Input (PDF/DOCX/TXT)
        ↓
File Parser Layer
        ↓
Text Preprocessing
        ↓
Feature Extraction
        ↓
```

```
Plagiarism Detection Model
        ↓
Scoring Engine
        ↓
Report Generator
```

---

4. Detailed Pipeline

Step 1: File Ingestion

- Accept formats:
- PDF → use PyMuPDF / pdfplumber
- DOCX → python-docx
- TXT → direct read

Step 2: Text Preprocessing

- Lowercasing
- Stopword removal
- Tokenization
- Lemmatization/Stemming

Step 3: Segmentation

- Split into:
- Sentences
- Paragraphs

Step 4: Feature Extraction

Options: - TF-IDF vectors - Word embeddings (Word2Vec, GloVe) - Sentence embeddings (BERT, SBERT)

Step 5: Model Training

Using PAN Dataset: - External plagiarism detection → supervised learning - Intrinsic plagiarism detection → anomaly detection

Models: - Logistic Regression - SVM - Random Forest - Transformer-based models (BERT)

Step 6: Similarity Computation

- Cosine similarity
- Jaccard similarity
- Semantic similarity (SBERT)

Step 7: Plagiarism Level Classification

- 0–30% → Low
- 30–70% → Medium
- 70–100% → High

Step 8: Report Generation

- Structured output
- Highlight plagiarized sections
- Provide percentage and classification

---

5. Shortcomings of Current Version

Technical Limitations

- Limited to text-based comparison
- No deep semantic understanding
- Poor paraphrase detection

Dataset Issues

- PAN 2009 is outdated
- Limited real-world diversity

System Gaps

- No multi-format ingestion (PDF/DOCX missing)
- No automated report generation
- No UI/dashboard

---

6. Final Vision and Goal

Vision

To build a production-grade plagiarism detection platform with: - Multi-format support - Deep semantic understanding - Real-time analysis

End Goals

- AI-powered plagiarism scoring
- Explainable results
- Visual analytics dashboard

---

7. Implementation Plan (Step-by-Step)

Phase 1: Data Preparation

1. Load PAN dataset
2. Parse XML metadata
3. Create labeled dataset

Phase 2: Preprocessing Pipeline

1. Build text cleaning module
2. Sentence segmentation
3. Store processed text

Phase 3: Baseline Model

1. Implement TF-IDF + Cosine similarity
2. Evaluate baseline performance

Phase 4: Advanced Model

1. Implement BERT/SBERT
2. Fine-tune on PAN dataset

Phase 5: File Handling System

1. Integrate PDF parser
2. Integrate DOCX parser
3. Normalize text output

Phase 6: Scoring Engine

1. Define plagiarism thresholds
2. Map similarity → levels

Phase 7: Report Generator

1. Create JSON output
2. Convert to human-readable report

Phase 8: UI (Optional)

- Dashboard using React/Streamlit

---

8. Report Structure (Final Output)

Section 1: Implementation

- Dataset description
- Model architecture
- Pipeline explanation

Section 2: Innovation

- Use of transformers
- Multi-format ingestion
- Semantic similarity scoring

Section 3: Analysis

- Accuracy
- Precision/Recall
- F1-score

---

9. Testing & Analysis Plan

Metrics

- Accuracy
- Precision
- Recall
- F1 Score

Experiments

1. Baseline vs Advanced model
2. Intrinsic vs External detection
3. File format comparison

---

10. Visualization Plan

Graphs to Generate

- Accuracy comparison bar chart
- Precision/Recall curves
- Confusion matrix heatmap

Tools

- Matplotlib
- Seaborn
- Plotly

---

11. Future Enhancements

- Add multilingual support
- Integrate web crawling for source detection
- Real-time plagiarism API

---

12. Software & Tools Required

Backend

- Python
- PyTorch / TensorFlow

Libraries

- scikit-learn
- transformers
- sentence-transformers

File Processing

- pdfplumber
- python-docx

Visualization

- matplotlib
- seaborn

---

13. Conclusion

This project aims to evolve from a basic plagiarism detection system into a robust AI-powered platform capable of handling real-world plagiarism scenarios across multiple formats with deep semantic understanding and detailed reporting.